TD1 : Wargame — Minimax avec coupures alpha-bêta

Teoman Delevaux — Master 1 I&ISC, CY Cergy Paris Université — 26 octobre 2026

1. Introduction et choix d'implémentation

L'objectif est de programmer une IA pour un wargame simplifié sur un plateau de 10×10. Chaque joueur déplace une unité d'une case par tour dans les 8 directions. Une unité ennemie est prise si la somme des valeurs alliées sur les 8 cases voisines est supérieure à celle des ennemis (en cas d'égalité, le défenseur gagne), et le perdant perd l'unité engagée. On gagne en atteignant la ligne de fond adverse ou en éliminant toutes les unités ennemies.

Algorithme. J'ai implémenté Minimax avec coupures alpha-bêta sous la forme de deux fonctions mutuellement récursives, f_max (tour de x) et f_min (tour de o). L'évaluation est toujours faite du point de vue de x (f_eval(plateau, 1)). f_IA joue le coup à la racine : elle teste chaque coup légal du joueur sur une copie du plateau, appelle f_min (si x joue) ou f_max (si o joue) pour les niveaux suivants, et garde le meilleur score (comparaison stricte, donc le premier coup trouvé l'emporte en cas d'égalité). Le jeu est ainsi déterministe.

Profondeur. La profondeur de recherche est de 3 demi-coups, comme demandé pour le tournoi : le coup de la racine, la réponse adverse, puis un coup du joueur. Elle est stockée dans une variable (prof_max) pour pouvoir la faire varier pendant les mesures.

Alpha-bêta. alpha est mis à jour dans f_max et beta dans f_min après chaque fils, et la boucle s'arrête dès que beta <= alpha. Le test est conditionné par une variable globale elagage, ce qui permet d'exécuter le même code en Minimax pur pour la comparaison.

Optimisation de l'énumération des coups. Un pion ne se déplace que d'une case. Les destinations sont donc limitées aux 8 cases voisines au lieu des 100 cases du plateau, ce qui divise le temps de calcul sans changer les coups explorés ni leur ordre.

Éléments ajoutés au code fourni (à signaler pour l'interconnexion des IA) :

- f_avance, f_centre, f_isoles : critères d'évaluation (section 2) ;
- f_bataille prend maintenant le plateau en paramètre (f_bataille(plateau, l, c)), car elle lisait le plateau global et jugeait donc les combats simulés sur le vrai plateau au lieu de la copie ;
- variables globales prof_max, elagage et nb_noeuds pour l'instrumentation, ainsi que f_afficher_stats, qui affiche en fin de partie le nombre de coups joués, le nombre moyen de nœuds visités et le temps moyen par coup de chaque IA.

2. Fonction d'évaluation

Pour un joueur *j*, avec (critère) = critère(*j*) − critère(−*j*) :

eval = 10*pions + 5*avancement + 2*matériel + 1*centralité − 5*isolés

Si un pion a atteint la ligne de fond adverse, f_eval renvoie ±INFINI (10000), qui écrase tout le reste.

Critère	Définition	Coef.	Justification
Nombre de pions	Nombre d'unités du joueur	10	Une prise est irréversible, et perdre toutes ses unités fait perdre la partie. C'est le critère le plus lourd.
Avancement	Somme des lignes parcourues vers la ligne adverse	5	Sans ce terme, l'évaluation reste constante tant qu'il n'y a pas de prise et l'IA n'a aucune raison de progresser vers l'objectif.
Matériel	Somme des valeurs des unités	2	La valeur compte dans `f_bataille` (somme des voisins). Coefficient faible car le nombre de pions capte déjà l'essentiel.
Centralité	$10 - \|2j - 9\|$ par pion, de 1 (bord) à 9 (colonnes E et F)	1	Un pion central a plus de voisins et de directions. Très faible poids : il sert à départager des coups sinon à égalité.
Pions isolés	Pions sans aucun allié sur leurs 8 voisins	−5	Comme le combat compare les sommes des voisins, un pion seul n'est soutenu par personne et perd facilement.

Hiérarchie des coefficients. Une prise change Δpions de 1 et le matériel de la valeur du pion, soit au moins 12 points, ce qui dépasse largement une avancée d'une case (5). Le malus d'isolement (−5) compense le gain d'avancement (+5) : une avancée qui isole un pion est à peu près neutre, alors qu'une avancée qui reste au contact des alliés est gratuite. L'IA avance donc plutôt en groupe.

Démarche. Les critères ont été ajoutés progressivement en observant des parties. Avec seulement le nombre de pions, le matériel et l'avancement, la majorité des coups avaient le même score. Comme f_IA garde le premier coup trouvé et balaie le plateau depuis la ligne 0, l'IA déplaçait toujours les pions du fond et formait des lignes. La centralité et les pions isolés ont été ajoutés pour départager ces coups. Les coefficients ont ensuite été réglés par essais, sans campagne systématique (voir section 4).

3. Analyse des coupures alpha-bêta

Protocole. Une partie IA contre IA (mode 3, profondeur 3) a été rejouée deux fois, avec elagage=1 puis elagage=0. L'IA étant déterministe, les deux exécutions suivent la même suite de positions, ce qui permet une comparaison appariée sur 40 coups (20 de x et 20 de o, la fin de la partie). Le compteur nb_noeuds est incrémenté à l'entrée de chaque appel de f_min et f_max (la racine n'est pas comptée) et le temps est mesuré avec clock() autour de f_IA.

Validation. Sur les 40 positions, le coup choisi et son score sont identiques avec et sans élagage (par exemple A2→A1 avec un score de −287, puis −323). L'élagage ne change donc pas le résultat, il évite seulement des calculs.

Résultats sur les 40 derniers coups.

	Sans élagage	Avec élagage	Gain
Nœuds, 40 coups	2 631 647	272 292	×9,7
Temps, 40 coups	5 167 ms	523 ms	×9,9
Nœuds par coup (moyenne)	65 791	6 807	×9,7
Joueur x : nœuds / temps	858 634 / 1 728 ms	240 394 / 459 ms	×3,6 / ×3,8
Joueur o : nœuds / temps	1 773 013 / 3 438 ms	31 898 / 64 ms	×55,6 / ×53,7

Le coût par nœud est stable, environ 2 µs dans les deux cas : le temps est proportionnel au nombre de nœuds et l'élagage n'ajoute pas de surcoût mesurable. Le pire coup observé sans élagage atteint 130 997 nœuds (255 ms), contre 22 089 nœuds (44 ms) au maximum avec élagage.

Pourquoi un gain si variable (×3,6 pour x, ×55 pour o) ? Dans cette fin de partie, un pion de o (3o en I8) est à une case de la ligne 9, donc o voit une victoire forcée (score −10000). Dès le premier coup examiné à la racine, beta vaut −10000, c'est-à-dire la valeur de alpha initial : la fenêtre est vide et presque tous les autres coups sont coupés immédiatement. C'est mon explication à partir du code, je n'ai pas mesuré la répartition des coupures. Pour x, qui perd, tous les coups valent −10000 et les coupures n'interviennent que sur la réponse gagnante de o, d'où un gain plus modeste. Le gain de ×55 est donc propre à ces positions à score terminal ; sur une partie entière (ci-dessous), l'écart entre joueurs est moins marqué.

Partie complète (mesure appariée). f_afficher_stats donne les moyennes sur une partie entière IA contre IA de 189 coups (95 de x, 94 de o), jouée avec puis sans élagage, à profondeur 3. Sur l'ensemble de la partie, l'élagage divise le nombre de nœuds par 10,4 et le temps par 10,0 : environ 15 600 nœuds et 36 ms par coup, contre environ 161 600 nœuds et 363 ms sans élagage.

Moyenne par coup	Sans élagage	Avec élagage	Gain
x : nœuds	155 948	18 891	×8,3
x : temps	356,0 ms	42,7 ms	×8,3
o : nœuds	167 258	12 282	×13,6
o : temps	370,1 ms	29,5 ms	×12,5

Le coût par nœud reste d'environ 2,3 µs avec ou sans élagage. Les totaux (environ 30,5 millions de nœuds contre 2,95 millions, 68,6 s contre 6,8 s) sont reconstitués à partir de ces moyennes. Ce facteur 10 sur une partie entière est cohérent avec celui des 40 derniers coups (×9,7) : il ne dépend donc pas seulement des positions terminales de la fin de partie.

Facteur de branchement effectif. Sur la partie complète, avec b^3 ≈ nombre de nœuds par coup, on obtient b ≈ 54 sans élagage (161 573^(1/3)) et b ≈ 25 avec (15 604^(1/3)). En extrapolant, une profondeur 4 coûterait environ 54 fois plus sans élagage (de l'ordre de 8,8 millions de nœuds, soit 20 s par coup) et environ 25 fois plus avec (de l'ordre de 390 000 nœuds, soit 0,9 s). C'est une estimation, pas une mesure, mais elle justifie la profondeur 3 du tournoi et l'intérêt de l'élagage.

Ordre d'exploration. Les coups sont parcourus dans l'ordre du plateau (ligne 0 vers 9, colonne A vers J), sans lien avec leur qualité. On est donc loin du meilleur cas de l'alpha-bêta (de l'ordre de b^(d/2) avec un tri parfait). Explorer d'abord les captures ou le meilleur coup de la recherche précédente augmenterait les coupures.

4. Observations et limites

Une IA gagnante qui ne conclut pas. Dans les deux exécutions, o joue 20 fois de suite A0 ↔ B0 alors que son pion en I8 pourrait atteindre la ligne 9 et gagner immédiatement. Tous les coups qui conservent la victoire forcée ont le même score (−10000) et f_IA garde le premier trouvé. À l'inverse, x, qui voit sa défaite, joue le premier coup trouvé et avance ses pions contre la ligne adverse, jusqu'à perdre sa dernière unité (la victoire de o vient d'ailleurs de l'élimination, pas de la ligne de fond). Il faut préférer les victoires rapides et les défaites lentes, par exemple en retirant la profondeur atteinte au score terminal (VICTOIRE − profondeur, avec VICTOIRE < INFINI pour rester dans la fenêtre alpha-bêta). C'est la première amélioration à apporter avant le tournoi.

Effet d'horizon. À profondeur 3, l'IA ne voit pas un plan de 5 demi-coups, par exemple une capture qui exige de déplacer deux pions pour soutenir le premier. Lors de mes premiers tests, PROF_MAX valait 2 et l'IA ne regardait en réalité que 2 demi-coups, ce qui lui a fait laisser passer un pion adverse jusqu'à sa ligne de fond. Rendre la profondeur variable et vérifier le nombre de demi-coups effectifs a corrigé ce point.

Victoire par élimination. Dans l'arbre, f_eval ne détecte que l'arrivée sur la ligne de fond. La perte de toutes les unités n'est pénalisée qu'indirectement, via le nombre de pions et le matériel.

Coefficients. Ils ont été choisis par essais sur quelques parties. Je n'ai pas fait de tournoi systématique entre plusieurs jeux de coefficients : une expérience plus rigoureuse ferait jouer deux versions l'une contre l'autre en mode IA contre IA, en échangeant les camps (x commence et a un léger avantage), et compterait les victoires.

5. Conclusion

L'implémentation Minimax avec alpha-bêta à profondeur 3 est fonctionnelle et choisit exactement les mêmes coups que le Minimax pur, avec environ 10 fois moins de nœuds et de temps sur la fin de partie étudiée (confirmé sur une partie complète de 189 coups : ×10,4 en nœuds et ×10,0 en temps). La fonction d'évaluation à cinq critères donne un jeu cohérent (prises, avance en groupe), mais elle a deux faiblesses identifiées : elle ne distingue pas une victoire rapide d'une victoire lente, et elle est limitée par l'horizon de 3 demi-coups. L'amélioration prioritaire est le score terminal dépendant de la profondeur, suivie d'un meilleur ordre d'exploration des coups.